สรุปภาพรวมเทคโนโลยีและสถาปัตยกรรมระบบ (Technology Stack)

แอปพลิเคชันรวมศูนย์ข้อความและการจัดการคำสั่งซื้อสำหรับธุรกิจออนไลน์ (Chatshop)

ผู้จัดทำ: นายธนพล วุฒิธาดา • อาจารย์ที่ปรึกษา: ผศ. ดร.อติราช สุขสวัสดิ์ • สถานประกอบการ: บจก. ไฮเฟนพลัส

วัตถุประสงค์เอกสาร: ใช้สำหรับเตรียมตัวนำเสนอโครงงานและเตรียมตอบข้อซักถามของคณะกรรมการสอบปริญญานิพนธ์ โดยสรุปครอบคลุมทุกหมวดหมู่ของเทคโนโลยี เหตุผลทางวิศวกรรมในการคัดเลือกเครื่องมือ และแนวทางตอบคำถามยอดฮิต

1. ฝั่งแอปพลิเคชันบนสมาร์ตโฟน (Frontend / Mobile App)

• Framework: Flutter (Dart SDK) พัฒนาแบบ Cross-Platform (มุ่งเน้นการทดสอบและใช้งานบน Android)

• Architectural Pattern: Feature-First / Layered Architecture แบ่งโครงสร้างโค้ดเป็น 4 เลเยอร์หลักตามหน้าที่ ได้แก่ core (บริการส่วนกลาง), data (Data Models & Repositories), features (13 โมดูลฟังก์ชัน), และ shared (Reusable Widgets & Bottom Sheets)

หมวดการทำงาน	ไลบรารี / เครื่องมือ	หน้าที่และเหตุผลการเลือกใช้ (Key Defense Arguments)
State Management	Riverpod (flutter_riverpod)	• จัดการ State ทั้งแอปพลิเคชันแบบ Compile-time Safe (ตรวจจับข้อผิดพลาดตั้งแต่ตอนเขียนโค้ด ไม่เกิด Runtime Error เรื่อง Provider Not Found) • แยก Business Logic ออกจาก UI อย่างเด็ดขาด • จุดเด่นสำคัญ: เมื่อผู้ใช้ออกจากระบบ (Logout) สามารถสั่ง Invalidate State ต้นทางเพื่อเคลียร์แคช, ยกเลิกการเชื่อมต่อ WebSocket และล้างโทเคนของผู้ใช้เดิมทิ้งได้ทันทีในคำสั่งเดียว
Navigation & Routing	go_router	• จัดการเส้นทางหน้าจอแบบ Declarative URL • รองรับ Deep Linking: เมื่อผู้ใช้แตะ Push Notification สามารถสั่งให้แอปเปิดตรงไปยังหน้าห้องแชท หรือหน้ารายการออเดอร์ของลูกค้ารายนั้นได้ทันที • Auto-Redirect: มีระบบตรวจสอบ Auth State หากระบบหลังบ้านตอบกลับรหัส 401 Unauthorized จะลบโทเคนและดีดกลับไปหน้า Login ทันที

หมวดการทำงาน	ไลบรารี / เครื่องมือ	หน้าที่และเหตุผลการเลือกใช้ (Key Defense Arguments)
Secure Storage	flutter_secure_storage	• จัดเก็บข้อมูลความลับ เช่น Bearer Token (JWT) และข้อมูลเซสชัน • ความปลอดภัยสูง: เข้ารหัสผ่านฮาร์ดแวร์โดยตรง (Android KeyStore และ iOS Keychain) ไม่ใช้ SharedPreferences ธรรมดา ป้องกันการถูกโจรกรรมข้อมูลเมื่ออุปกรณ์ถูก Root
Network & API Client	Custom HTTP Client (REST API)	• เรียกใช้งาน REST API กว่า 99 เส้นทางของระบบหลังบ้าน • มี Interceptor คอยแนบ Bearer Token ให้อัตโนมัติทุก Request • รองรับการส่งข้อมูลแบบ Multipart Request สำหรับการอัปโหลดไฟล์ รูปภาพ วิดีโอ และสลิปการโอนเงิน
Real-time Client	WebSocket Client (Pusher Protocol)	• เชื่อมต่อไปยัง Laravel Reverb ผ่านโปรโตคอล Pusher • รับเหตุการณ์แบบเรียลไทม์: ข้อความใหม่, อัปเดตสถานะการอ่าน, การล็อกบทสนทนา, ออเดอร์เปลี่ยนสถานะ และ Typing Indicator (กำลังพิมพ์...)
Push Notification	firebase_messaging flutter_local_notifications	• เชื่อมต่อ Firebase Cloud Messaging (FCM) • รับการแจ้งเตือนข้อความใหม่และแชทค้างตอบได้ทันทีแม้ปิดแอปพลิเคชันอยู่ • แสดงผลเป็น Heads-up Banner พร้อมปุ่มเปิดห้องแชทเมื่อผู้ใช้กำลังเปิดแอปอยู่
Social Authentication	google_sign_in flutter_facebook_auth	• จัดการการเข้าสู่ระบบด้วยบัญชี Google และ Facebook จากหน้า Native ของสมาร์ตโฟน แล้วส่ง Token ไปตรวจสอบความถูกต้องกับระบบหลังบ้าน

2. ฝั่งระบบบริการหลังบ้าน (Backend Server Engine)

• Core Framework: Laravel 10 บนสภาพแวดล้อม PHP 8.2+ (PHP-FPM)

• Web Server & Security: รันภายใต้ Nginx Reverse Proxy พร้อมใบรับรองความปลอดภัย SSL/TLS (HTTPS) ผ่าน Let's Encrypt ซึ่งจำเป็นอย่างยิ่งเพราะ LINE และ Meta บังคับให้ Webhook URL ต้องเป็น HTTPS เท่านั้น

องค์ประกอบ	เทคโนโลยีที่ใช้	รายละเอียดการทำงานเชิงเทคนิค

องค์ประกอบ	เทคโนโลยีที่ใช้	รายละเอียดการทำงานเชิงเทคนิค
API & Authentication	Laravel Sanctum + HMAC-SHA256	• ออก Bearer API Token (Personal Access Token) ให้กับแอปพลิเคชันมือถือ • ตรวจสอบความถูกต้องของ Social Token จาก Google/Facebook ก่อนออก Token ของระบบ • Webhook Security: ตรวจสอบลายเซ็นดิจิทัลของแพลตฟอร์ม LINE และ Meta ด้วยอัลกอริทึม HMAC-SHA256 ป้องกันการปลอมแปลง Request
Real-time Engine	Laravel Reverb (WebSocket Server)	• เซิร์ฟเวอร์ WebSocket ประสิทธิภาพสูงของ Laravel รันบน Event Loop • Broadcast เหตุการณ์สำคัญ เช่น `NewMessage`, `OrderUpdated`, `SettingsChanged` สู่แอปพลิเคชันระดับมิลลิวินาที โดยไม่ต้องเสียค่าบริการ Third-party Cloud
Queue & Workers	Laravel Queue (Supervisor Control)	• สถาปัตยกรรม Asynchronous Queue แบ่งเป็น 3 คิวตามความสำคัญ: 1. `high`: ประมวลผล Webhook ขาเข้าแบบด่วน 2. `default`: งานทั่วไป เช่น อัปเดตข้อมูลลูกค้าและออเดอร์ 3. `typing`: จำลองสถานะการพิมพ์ของแชทบอท • ควบคุมกระบวนการทำงานด้วย Supervisor เพื่อสั่ง Auto-Restart ทันทีหากโปรเซสหยุดทำงาน
Task Scheduler	Laravel Scheduler (Linux Cron 1 min)	• รันเบื้องหลังทุก 1 นาที ตรวจหาบทสนทนาที่ค้างตอบเกิน SLA ที่ร้านตั้งไว้ (5, 10, 15, 20 นาที) และส่ง Push Notification ไปยังเจ้าหน้าที่ผู้รับผิดชอบ
Media Processing	PHP GD & FFmpeg	• ปรับขนาดและบีบอัดรูปภาพด้วย PHP GD Library • แปลงและลดขนาดไฟล์วิดีโอด้วย FFmpeg ก่อนจัดเก็บลงดิสก์ ช่วยลดปริมาณพื้นที่และการใช้แบนด์วิดท์

3. สถาปัตยกรรมฐานข้อมูลแบบผสมผสาน (Hybrid / Dual Database Architecture)

ระบบออกแบบโครงสร้างจัดเก็บข้อมูลแบบ Polyglot Persistence โดยเลือกใช้ฐานข้อมูล 2 ประเภทควบคู่กันตามธรรมชาติของข้อมูล:

ฐานข้อมูล	ประเภท	ข้อมูลที่จัดเก็บ	เหตุผลทางวิศวกรรม (Engineering Rationale)
MySQL (InnoDB)	Relational Database (RDBMS)	• ร้านค้า (shops) • ผู้ใช้งาน (users) • สิทธิ์และบทบาท (roles/RBAC) • สมาชิกทีม (team) • แคตตาล็อกสินค้า (products) • รายการคำสั่งซื้อ (orders)	• ต้องการความถูกต้องแม่นยำสูง (ACID Compliance): มีระบบ Transaction และ Foreign Keys แบบลบตามกัน (Cascade Delete) ป้องกันข้อมูลค้างเมื่อลบร้านค้า • เหมาะกับข้อมูลเชิงโครงสร้างที่มีความสัมพันธ์แน่นอน เช่น ข้อมูลการเงินและสต็อกสินค้า • ใช้ชุดอักขระ `utf8mb4` รองรับภาษาไทยและ Emoji สมบูรณ์
MongoDB (Document)	NoSQL (Document Store)	• ข้อความแชท (`messages`) • บทสนทนา (`conversations`) • เอกสารความรู้ (`knowledge_documents`) • เวกเตอร์ Embeddings (`knowledge_chunks`) • โทเคนอุปกรณ์ (`device_tokens`)	• โครงสร้างยืดหยุ่นสูง (Flexible Schema): ข้อความแชทจาก 4 แพลตฟอร์มมี Payload ต่างกันมาก (ข้อความ, รูปภาพ, สติกเกอร์, พิกัด, ปุ่มกด) • รองรับปริมาณมหาศาล (High Write Throughput): ข้อความแชทเพิ่มขึ้นตลอดเวลา MongoDB อ่านและเขียนข้อมูลอาร์เรย์ในเอกสารเดียวได้รวดเร็วโดยไม่ต้อง Join ตาราง

4. ปัญญาประดิษฐ์และระบบ RAG (AI, LLM Integration & RAG Architecture)

ส่วนประกอบ	เทคโนโลยี / โมเดล	รายละเอียดและกลไกการทำงาน
Multi-LLM Provider	• Ollama (Local/On-Prem) • Google Gemini API • Anthropic Claude • OpenAI API	• ระบบหลังบ้านออกแบบเป็น Interface รองรับการสลับผู้ให้บริการโมเดลภาษาได้อิสระจากหน้าจอตั้งค่าของแอปพลิเคชัน • ร้านค้าเลือกเชื่อมต่อ Local LLM เพื่อความเป็นส่วนตัวของข้อมูล หรือใช้ Cloud LLM เพื่อความฉลาดในการตอบ

ส่วนประกอบ	เทคโนโลยี / โมเดล	รายละเอียดและกลไกการทำงาน
RAG Knowledge Base	Text Chunking + Dense Embeddings + Cosine Similarity	• Chunking: นำเอกสารคู่มือ/สินค้าของร้านมาตัดแบ่งเป็นท่อนย่อย • Embedding: แปลงเนื้อหาเป็น Dense Vector บันทึกลงในคอลเลกชัน `knowledge_chunks` ของ MongoDB • Retrieval: เมื่อลูกค้าทักถาม ระบบแปลงคำถามเป็นเวกเตอร์และคำนวณความคล้ายโคไซน์ (Cosine Similarity) เพื่อดึงเฉพาะเนื้อหาที่เกี่ยวข้องส่งไปเป็นบริบท (Context) ให้ AI ตอบ
Chatbot Control Modes	1. Manual (คนตอบ 100%) 2. Suggest (AI เสนอคำตอบ) 3. Autopilot (AI ตอบอัตโนมัติ)	• เจ้าหน้าที่สามารถปรับโหมดการทำงานของแชทบอทแยกตามแต่ละบทสนทนาได้ทันทีตามสถานการณ์จริง • ในโหมด Autopilot หากแชทบอทตอบไม่ได้ ระบบจะตัดเข้าโหมดส่งต่อเจ้าหน้าที่และแจ้งเตือนทันที
Conversation Analytics	LLM Prompt Pipeline	• วิเคราะห์สรุปบทสนทนา (Summary) • จำแนกอารมณ์ลูกค้า (Sentiment: Positive, Neutral, Negative) • วิเคราะห์เจตนา (Intent) และตรวจจับสัญญาณสำคัญ (เช่น ลูกค้าเร่งด่วน, พร้อมชำระเงิน)

5. การเชื่อมต่อแพลตฟอร์มภายนอกและโครงสร้างพื้นฐาน (Integration & DevOps)

หมวดหมู่	แพลตฟอร์ม / เครื่องมือ	แนวทางการประยุกต์ใช้งานจริง
LINE Integration	LINE Messaging API (Channel Access Token v2.1)	• ร้านค้าเชื่อมต่อด้วย Channel ID และ Channel Secret • รับ Webhook ขาเข้า และส่งข้อความขาออกผ่าน LINE Push Message API • ปิดบอทอัตโนมัติของ LINE Official Account Manager เพื่อให้ระบบ Chatshop ควบคุมแต่เพียงผู้เดียว

หมวดหมู่	แพลตฟอร์ม / เครื่องมือ	แนวทางการประยุกต์ใช้งานจริง
Meta Integration	Meta Graph API (v19.0) (Facebook & Instagram)	• แยกเป็น 2 Meta Apps ในระบบ: 1. *Consumer App:* สำหรับ Facebook Login ของผู้ใช้ทั่วไป 2. *Business App:* สำหรับรับ Webhook และส่งข้อความผ่าน Facebook Page & Instagram Messaging • แก้ปัญหาการขอสิทธิ์ขั้นสูง (Advanced Permissions) ทำให้ผู้ใช้ที่ไม่มีเพจสามารถเข้าใช้งานแอปได้
WebChat Integration	JavaScript Embed Widget	• สคริปต์วิดเจ็ตขนาดเล็กสำหรับนำไปติดบนหน้าเว็บไซต์ของร้านค้าเพื่อเปิดกล่องแชทสดคุยเชื่อมโยงเข้ามายังแอป Chatshop
CI/CD Pipeline	Bitbucket Pipelines (Automated Deployment)	• เมื่อรวมโค้ดเข้าสาขา `release` ระบบจะรันชุดทดสอบ ติดตั้งไลบรารี และ Deploy ไฟล์ขึ้นเซิร์ฟเวอร์ผ่าน SSH ด้วย `rsync` อัตโนมัติ โดยไม่เขียนทับไฟล์ `.env` และพื้นที่เก็บรูปภาพ
Architecture Design	Multi-Tenant Architecture & RBAC Security	• Multi-Tenant: แยกข้อมูลของแต่ละร้านค้าอย่างเด็ดขาดด้วย `shop_id` ทุกคำสั่งค้นหา ป้องกันปัญหาข้อมูลรั่วไหลข้ามร้าน • RBAC: ควบคุมสิทธิ์การเข้าถึงเมนูและฟังก์ชันตามบทบาท (Owner, Admin, Member)

6. แนวคำถาม-คำตอบเก็งข้อสอบสำหรับป้องกันโครงงาน (Defense Q&A Cheat Sheet)

Q1: ทำไมเลือกพัฒนาแอปด้วย Flutter แทนที่จะเป็น Native (Kotlin บน Android / Swift บน iOS)?

A: เนื่องจากระบบ Chatshop ต้องการรองรับผู้ใช้งานทั้งบน Android และ iOS ในอนาคต การเลือกใช้ Flutter ช่วยให้สามารถพัฒนาบนฐานโค้ดชุดเดียวกัน (Single Codebase) แต่ได้ประสิทธิภาพการทำงานและการวาดผลหน้าจอในระดับเนทีฟ มีเอนจินกราฟิกของตัวเองทำให้ UI แสดงผลเหมือนกันทุกรุ่น ทุกยี่ห้อสมาร์ตโฟน และช่วยลดระยะเวลาในการพัฒนาลงได้อย่างมีนัยสำคัญครับ

Q2: ทำไมเลือกใช้ Riverpod ในการจัดการ State แทนที่จะเป็น Bloc หรือ GetX?

A: Riverpod ได้รับการพัฒนาขึ้นเพื่อแก้ข้อจำกัดของ Provider เดิม โดยทำงานเป็นอิสระอยู่นอก Widget Tree ทำให้สามารถจัดการ Global Services เช่น การสั่งตัดการเชื่อมต่อ WebSocket หรือการล้างโทเคนเมื่อ Logout ได้อย่างสะดวกมาก และที่สำคัญที่สุดคือ Riverpod มีความปลอดภัยสูงมากเพราะตรวจจับข้อผิดพลาดเรื่อง Type และ Dependency ตั้งแต่ขั้นตอน Compile (Compile-time Safety) ป้องกันการเกิดข้อผิดพลาด Runtime Error ระหว่างผู้ใช้เปิดแอปครับ

Q3: ทำไมฐานข้อมูลต้องใช้ทั้ง MySQL และ MongoDB ร่วมกัน ทำไมไม่ใช้ตัวใดตัวหนึ่ง?

A: เพราะลักษณะของข้อมูลในระบบมีความต้องการทางวิศวกรรมที่แตกต่างกันอย่างสิ้นเชิงครับ

1) ข้อมูลร้านค้า สมาชิก สินค้า และคำสั่งซื้อ เป็นข้อมูลเชิงโครงสร้างที่ต้องการความถูกต้องแม่นยำและเสถียรภาพทางการ

เงินระดับสูงสุด (ACID Compliance) จึงใช้ MySQL ที่มี Foreign Keys และ Transaction คอยควบคุม

2) ในขณะที่ข้อความแชทจาก 4 แพลตฟอร์มมีโครงสร้างไม่แน่นอน (ข้อความ, รูปภาพ, สติกเกอร์, พิกัด) และมีปริมาณไหลเข้าสู่ระบบอย่างต่อเนื่องมหาศาล MongoDB ซึ่งเป็น Document NoSQL ตอบโจทย์เรื่องความยืดหยุ่นของฟิลด์และมีประสิทธิภาพในการอ่านเขียนข้อมูลปริมาณมากได้เร็วกว่าการ Join ตารางใน RDBMS ครับ

Q4: หากมีลูกค้ากดแชทเข้ามาพร้อมกันจากหลายช่องทางจำนวนมาก เซิร์ฟเวอร์จะค้างหรือล่มหรือไม่?

A: ไม่ค้างครับ เพราะระบบออกแบบเป็น Asynchronous Queue Architecture ทันทีที่ Webhook จาก LINE หรือ Meta วิ่งเข้ามา ระบบหลังบ้านจะทำเพียงแค่ตรวจสอบความถูกต้องของลายเซ็นดิจิทัล (HMAC) แล้วผลักงานเข้าคิว High ทันที พร้อมกับส่งการตอบกลับ HTTP Status 200 กลับไปหาแพลตฟอร์มต้นทางภายในเวลาไม่กี่มิลลิวินาที แพลตฟอร์มจึงไม่เกิดการส่งข้อความซ้ำ จากนั้น Worker ในเบื้องหลังจะทยอยดึงงานไปประมวลผลและส่งต่อผ่าน WebSocket และ FCM ไปยังแอปพลิเคชันอย่างราบรื่นครับ

Q5: ทำไมระบบแชทบอทต้องทำกระบวนการ RAG เอง ทำไมไม่ส่งเอกสารทั้งหมดไปใน Prompt เลย?

A: เพราะข้อมูลแคตตาล็อกสินค้าและเงื่อนไขการให้บริการของร้านค้ามีขนาดยาวมาก หากส่งเนื้อหาทั้งหมดไปในทุกคำขอ จะทำให้สิ้นเปลืองค่าใช้จ่ายโทเคน (Token Cost) สูงมาก และโมเดลภาษาอาจเกิดอาการภาพหลอน (Hallucination) หรือหลงลืมเนื้อหาตรงกลาง การใช้ RAG ด้วยการทำ Chunking และคำนวณความคล้ายโคไซน์ (Cosine Similarity) ทำให้เราส่งเฉพาะส่วนย่อยของเนื้อหาที่ตรงกับคำถามของลูกค้าที่สุดไปเป็นบริบท ทำให้ AI ตอบได้อย่างแม่นยำ ตรงประเด็น และประหยัดค่าใช้จ่าย API ได้ถึง 80-90% ครับ